

Doc 10.3 — Rest Parameters & Arguments Object

এই ডকে আমরা শিখব কীভাবে একটি function যেকোনো সংখ্যক **arguments** গ্রহণ করতে পারে।

মূল topics:

- Rest Parameters `...args`
- Variable number of arguments
- Rest parameter syntax
- Rest parameter must be last
- Rest parameter কীভাবে array হিসেবে কাজ করে
- Rest vs Spread
- `arguments` object
- `arguments.length`
- `arguments` বনাম Rest Parameters
- Arrow function এবং `arguments`
- `arguments` → Array conversion
- Modern JavaScript best practice

1. Function-এ Arguments কী?

Function define করার সময় আমরা parameters লিখি:

```
function add(a, b) {  
  return a + b;  
}
```

Call করার সময় values পাঠাই:

```
add(10, 20);
```

এখানে:

```
a → 10  
b → 20
```

কিন্তু যদি আমরা জানি না কয়টা value আসবে?

যেমন:

```
sum(10, 20);  
sum(10, 20, 30);  
sum(10, 20, 30, 40, 50);
```

তখন **Rest Parameter** খুব useful।

2. Rest Parameter কী?

Rest parameter হলো:

```
...args
```

এটি function-এর বাকি arguments-গুলোকে একটি **array**-এর মধ্যে **collect** করে।

Example:

```
function sum(...numbers) {  
  console.log(numbers);  
}  
  
sum(10, 20, 30, 40);
```

Output:

```
[10, 20, 30, 40]
```

অর্থাৎ:

```
arguments  
  ↓  
...numbers  
  ↓  
Array
```

3. Rest Parameter-এর Basic Syntax

```
function functionName(...args) {  
  // args is an array  
}
```

Example:

```
function showNumbers(...numbers) {  
  console.log(numbers);  
}  
  
showNumbers(1, 2, 3, 4);
```

Result:

```
[1, 2, 3, 4]
```

4. Rest Parameter-এর নাম `args` হওয়া বাধ্যতামূলক নয়

অনেকে ভাবে:

```
...args
```

হতেই হবে।

না।

যেকোনো `valid variable name` ব্যবহার করা যায়।

```
function test(...values) {  
  console.log(values);  
}
```

অথবা:

```
function test(...numbers) {  
  console.log(numbers);  
}
```

অথবা:

```
function test(...items) {  
  console.log(items);  
}
```

সব valid ।

5. Rest Parameter আসলে Array

এটা খুব important ।

```
function test(...values) {  
  console.log(Array.isArray(values));  
}  
  
test(10, 20, 30);
```

Output:

```
true
```

কারণ:

```
...values  
  ↓  
Array
```

6. Rest Parameter দিয়ে Sum

সব arguments যোগ করতে পারি:

```
function sum(...numbers) {  
  let total = 0;  
  
  for (const number of numbers) {  
    total += number;  
  }  
  
  return total;  
}  
  
console.log(sum(10, 20, 30));
```

Output:

60

7. Rest Parameter + `reduce()`

আরও concise:

```
function sum(...numbers) {  
  return numbers.reduce((total, number) => total + number, 0);  
}  
  
console.log(sum(10, 20, 30, 40));
```

Output:

100

এখানে:

```
...numbers  
  ↓  
array  
  ↓  
reduce()
```

↓
sum

8. Variable Number of Arguments

Rest parameter-এর সবচেয়ে বড় সুবিধা হলো function-এর arguments সংখ্যা fixed না হলেও কাজ করতে পারে।

```
function sum(...numbers) {  
  return numbers.reduce((sum, number) => sum + number, 0);  
}
```

এখন:

```
sum();
```

Result:

```
0
```

```
sum(10);
```

Result:

```
10
```

```
sum(10, 20, 30);
```

Result:

```
60
```

9. Rest Parameter-এর আগে Normal Parameters থাকতে পারে

হ্যাঁ।

```
function greet(greeting, ...names) {  
  console.log(greeting);  
  console.log(names);  
}
```

Call:

```
greet("Hello", "Rahim", "Karim", "Shariar");
```

Output:

```
Hello  
["Rahim", "Karim", "Shariar"]
```

এখানে:

```
greeting → "Hello"  
  
names → ["Rahim", "Karim", "Shariar"]
```

10. Rest Parameter অবশ্যই Last Parameter হতে হবে

সঠিক:

```
function test(a, b, ...rest) {  
  console.log(a, b, rest);  
}
```

ভুল:

```
function test(...rest, a, b) {  
  // SyntaxError  
}
```

কারণ rest parameter সবসময় শেষে থাকবে।

11. কেন Rest Parameter Last হতে হয়?

কারণ JavaScript-কে জানতে হবে:

কোন arguments normal parameters-এর জন্য?
কোন arguments rest-এর মধ্যে যাবে?

Example:

```
function test(a, b, ...rest) {}
```

JavaScript বুঝতে পারে:

প্রথম argument → a
দ্বিতীয় argument → b
বাকি সব → rest

12. Rest Parameter + Normal Parameters

```
function multiply(multiplier, ...numbers) {  
  return numbers.map(number => number * multiplier);  
}
```

Call:

```
console.log(multiply(2, 10, 20, 30));
```

Output:

```
[20, 40, 60]
```

এখানে:

```
multiplier → 2
```

```
numbers → [10, 20, 30]
```

13. Rest Parameter Empty হতে পারে

```
function test(...args) {  
  console.log(args);  
}  
  
test();
```

Output:

```
[]
```

এটা `undefined` নয়।

Rest parameter সবসময় array।

Arguments না থাকলে empty array:

```
[]
```

14. Rest Parameter-এর Array Methods ব্যবহার করা যায়

কারণ এটি array:

```
function test(...numbers) {  
  console.log(numbers.length);  
  console.log(numbers.map(number => number * 2));  
}
```

```
    console.log(numbers.filter(number => number > 10));  
  }
```

Call:

```
test(5, 10, 15, 20);
```

এখানে সরাসরি:

```
map()  
filter()  
reduce()  
includes()  
indexOf()  
forEach()
```

ইত্যাদি ব্যবহার করতে পারবে।

15. Rest Parameter + Arrow Function

Arrow function-এর সাথেও rest parameter ব্যবহার করা যায়:

```
const sum = (...numbers) => {  
  return numbers.reduce((sum, number) => sum + number, 0);  
};
```

Example:

```
console.log(sum(10, 20, 30));
```

Output:

```
60
```

16. Rest Parameter + Implicit Return

আরও short:

```
const sum = (...numbers) =>
  numbers.reduce((sum, number) => sum + number, 0);
```

এখানে:

```
...numbers
  ↓
array
  ↓
reduce
  ↓
implicit return
```

17. Rest vs Spread — সবচেয়ে Important

দুটোতেই `...` ব্যবহার হয়।

কিন্তু কাজ আলাদা।

Rest

```
function test(...args) {}
```

Multiple values → একটি array

Spread

```
const numbers = [1, 2, 3];

console.log(...numbers);
```

একটি **iterable** → **multiple values**

সহজভাবে:

REST
multiple → one

SPREAD
one → multiple

18. Rest Example

```
function test(...values) {  
  console.log(values);  
}  
  
test(10, 20, 30);
```

Result:

```
[10, 20, 30]
```

19. Spread Example

```
const numbers = [10, 20, 30];  
  
Math.max(...numbers);
```

এখানে:

```
...numbers
```

array-এর values আলাদা arguments হিসেবে পাঠাচ্ছে:

```
Math.max(10, 20, 30);
```

20. Rest vs Spread Mental Model

মনে রাখো:

REST

↓

"বাকি সব নিয়ে একটা collection বানাও"

SPREAD

↓

"collection-টা খুলে values ছড়িয়ে দাও"

21. arguments Object কী?

Rest parameter আসার আগে function-এর সব arguments access করার জন্য JavaScript-এ built-in:

arguments

ব্যবহার করা হতো।

Example:

```
function test() {  
  console.log(arguments);  
}  
  
test(10, 20, 30);
```

Output-এর মতো:

[Arguments] { 0: 10, 1: 20, 2: 30 }

এটা দেখতে array-এর মতো হলেও সাধারণ **Array** নয়।

22. arguments হলো Array-like Object

খুব important:

arguments

Array-এর মতো index access করতে পারে:

```
function test() {  
  console.log(arguments[0]);  
  console.log(arguments[1]);  
}  
  
test("A", "B");
```

Output:

A
B

কিন্তু:

Array.isArray(arguments)

সাধারণত:

false

23. arguments.length

Arguments কয়টি এসেছে সেটা দেখতে:

```
function test() {  
  console.log(arguments.length);  
}  
  
test(10, 20, 30, 40);
```

Output:

4

24. arguments দিয়ে Sum

পুরোনো style:

```
function sum() {  
  let total = 0;  
  
  for (let i = 0; i < arguments.length; i++) {  
    total += arguments[i];  
  }  
  
  return total;  
}
```

Call:

```
sum(10, 20, 30);
```

Result:

60

25. Rest Parameter দিয়ে একই কাজ

Modern JavaScript:

```
function sum(...numbers) {  
  let total = 0;  
  
  for (const number of numbers) {  
    total += number;  
  }  
}
```

```
    return total;
}
```

এটা অনেক cleaner।

কারণ:

```
arguments
→ array-like object

rest parameter
→ real array
```

26. arguments vs Rest Parameter

Feature	arguments	Rest Parameter
Type	Array-like object	Array
Array methods	Directly সব নেই	আছে
Modern approach	✗ পুরোনো style	✓ Preferred
Arrow function	নিজস্ব নেই	আছে
Naming	Fixed arguments	নিজের নাম দিতে পারো
Selective parameters	Less clear	Much clearer
Readability	তুলনামূলক কম	বেশি

27. Rest Parameter কেন Better?

Rest:

```
function sum(...numbers) {
    return numbers.reduce((a, b) => a + b, 0);
}
```

এখানে instantly বোঝা যাচ্ছে:

```
numbers = array of arguments
```

`arguments` ব্যবহার করলে:

```
function sum() {  
  // What does arguments contain?  
}
```

Function signature দেখে ততটা clear হয় না।

28. Arrow Function এবং `arguments`

এটা খুব important।

Arrow function-এর নিজের `arguments` object নেই।

```
const test = () => {  
  console.log(arguments);  
};
```

এখানে `arguments` arrow function-এর নিজস্ব নয়।

29. Arrow Function-এ Multiple Arguments দরকার হলে Rest ব্যবহার করো

ভুল approach:

```
const sum = () => {  
  console.log(arguments);  
};
```

Better:

```
const sum = (...numbers) => {  
  return numbers.reduce((a, b) => a + b, 0);  
};
```

এটাই modern এবং clean approach ।

30. Regular Function-এ arguments

Regular function:

```
function test() {  
  console.log(arguments);  
}  
  
test(1, 2, 3);
```

নিজস্ব arguments পাওয়া যায় ।

31. Arrow Function vs Regular Function

```
function regular() {  
  console.log(arguments);  
}
```

Regular function:

নিজস্ব arguments → আছে

Arrow:

```
const arrow = () => {  
  // no own arguments  
};
```

Arrow function:

নিজস্ব arguments → নেই

32. arguments থেকে **Array** বানানো

Legacy code-এ arguments থেকে Array বানানোর প্রয়োজন হতে পারে।

Array.from()

```
function test() {  
  const args = Array.from(arguments);  
  
  console.log(args);  
}  
  
test(10, 20, 30);
```

Result:

[10, 20, 30]

33. Spread দিয়ে arguments → Array

আরেকটি উপায়:

```
function test() {  
  const args = [...arguments];  
  
  console.log(args);  
}
```

এখন:

arguments
↓
spread

↓
Array

34. Rest Parameter দিয়ে Conversion-এর দরকার নেই

যদি function তুমি নিজে লিখছো:

```
function test(...args) {  
  console.log(args);  
}
```

এখানে conversion দরকার নেই।

কারণ `args` শুরু থেকেই array।

35. Rest Parameter + Destructuring

Rest parameter normal parameters-এর সাথে combine করা যায়।

```
function test(first, second, ...others) {  
  console.log(first);  
  console.log(second);  
  console.log(others);  
}  
  
test(10, 20, 30, 40, 50);
```

Output:

```
10  
20  
[30, 40, 50]
```

36. Practical Example — Shopping Cart

ধরো cart-এ যেকোনো সংখ্যক product price আসতে পারে:

```
function calculateTotal(...prices) {  
  return prices.reduce((total, price) => total + price, 0);  
}  
  
console.log(calculateTotal(500, 300, 700));
```

Result:

```
1500
```

37. Practical Example — Tags

```
function createPost(title, ...tags) {  
  return {  
    title,  
    tags  
  };  
}  
  
console.log(  
  createPost("JavaScript Basics", "js", "frontend", "coding")  
);
```

Result:

```
{  
  title: "JavaScript Basics",  
  tags: ["js", "frontend", "coding"]  
}
```

38. Practical Example — Logging

```
function logMessage(level, ...messages) {  
  console.log(level, messages);  
}
```

```
logMessage("ERROR", "Database", "connection", "failed");
```

এখানে:

```
level
↓
"ERROR"

messages
↓
["Database", "connection", "failed"]
```

39. Practical Example — Maximum Value

```
function getMax(...numbers) {
  return Math.max(...numbers);
}

console.log(getMax(10, 50, 20, 80, 30));
```

Result:

80

খোঁজ করো এখানে `...` দুই জায়গায় এসেছে কিন্তু কাজ আলাদা:

```
function getMax(...numbers)
```

এখানে **Rest** ।

আর:

```
Math.max(...numbers)
```

এখানে **Spread** ।

40. Rest + Spread একই Function-এ

এই example খুব important:

```
function getMax(...numbers) {  
  return Math.max(...numbers);  
}
```

প্রথম:

```
...numbers
```

→ Rest

দ্বিতীয়:

```
...numbers
```

→ Spread

Flow:

```
10, 20, 30, 40  
  ↓  
Rest  
  ↓  
[10, 20, 30, 40]  
  ↓  
Spread  
  ↓  
Math.max(10, 20, 30, 40)
```

41. Rest Parameter + `map()`

```
function doubleAll(...numbers) {  
  return numbers.map(number => number * 2);  
}
```

```
console.log(doubleAll(1, 2, 3, 4));
```

Output:

```
[2, 4, 6, 8]
```

42. Rest Parameter + Filter

```
function getEven(...numbers) {  
  return numbers.filter(number => number % 2 === 0);  
}  
  
console.log(getEven(1, 2, 3, 4, 5, 6));
```

Output:

```
[2, 4, 6]
```

43. Rest Parameter + Object

Rest parameter function-এর arguments collect করে array-তে।

কিন্তু object destructuring-এর rest pattern আলাদা concept:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "developer"  
};  
  
const { name, ...others } = user;
```

এখানে:

```
name
↓
"Shariar"

others
↓
{
  age: 31,
  role: "developer"
}
```

এটাও `...` ব্যবহার করে, কিন্তু এটা **object rest pattern**।

44. Rest Parameter vs Object Rest Pattern

Function:

```
function test(...args) {}
```

এখানে:

Rest Parameter

Object:

```
const { name, ...others } = user;
```

এখানে:

Rest Pattern

Array:

```
const [first, ...others] = numbers;
```

এখানেও:

Rest Pattern

সবগুলোর common idea:

বাকি অংশ **collect** করা।

45. Rest vs Spread vs Destructuring Rest

এক নজরে:

Syntax	Context	কাজ
<code>function f(...args)</code>	Function parameter	Arguments collect করে
<code>[first, ...rest]</code>	Array destructuring	বাকি values collect করে
<code>{name, ...rest}</code>	Object destructuring	বাকি properties collect করে
<code>[...arr]</code>	Array expression	Values spread করে
<code>{...obj}</code>	Object expression	Properties spread করে
<code>fn(...arr)</code>	Function call	Values arguments হিসেবে spread করে

46. Common Mistake — Rest Parameter মাঝখানে

ভুল:

```
function test(...args, last) {  
}
```

এটা `SyntaxError`।

সঠিক:

```
function test(first, ...args) {  
}
```

47. Common Mistake — Rest আর Spread একই ভাষা

```
function test(...args) {}
```

এটা Rest ।

কিন্তু:

```
test(...values);
```

এটা Spread ।

Context দেখেই বুঝতে হবে ।

48. Common Mistake — Rest Parameter Object নয়

```
function test(...args) {  
  console.log(args);  
}
```

`args` হলো Array:

```
[10, 20, 30]
```

Object নয় ।

49. Common Mistake — `arguments` কে Array ভাষা

```
function test() {  
  console.log(Array.isArray(arguments));  
}
```

Result:

false

arguments হলো array-like object।

50. Common Mistake — Arrow Function-এ

arguments

```
const test = () => {  
  console.log(arguments);  
};
```

এখানে arrow function-এর own arguments নেই।

Multiple arguments দরকার হলে:

```
const test = (...args) => {  
  console.log(args);  
};
```

51. arguments কখন দেখতে পেতে পারি?

Modern code-এ নতুন function লিখলে Rest Parameter prefer করা ভালো।

তবে পুরোনো JavaScript codebase, legacy libraries, বা existing functions-এ:

arguments

দেখতে পাবে।

তাই এটি **understand** করা দরকার, যদিও নতুন code-এ সবসময় ব্যবহার করা দরকার নেই।

52. Output Practice

Question 1

```
function test(...args) {  
  console.log(args);  
}  
  
test(10, 20, 30);
```

Output কী?

Question 2

```
function test(...args) {  
  console.log(args.length);  
}  
  
test("a", "b", "c", "d");
```

Output কী?

Question 3

```
function test(first, ...rest) {  
  console.log(first);  
  console.log(rest);  
}  
  
test(10, 20, 30, 40);
```

Output কী?

Question 4

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}
```

```
console.log(sum(5, 10, 15));
```

Output কী?

Question 5

```
function test() {  
  console.log(Array.isArray(arguments));  
}  
  
test(1, 2, 3);
```

Output কী?

Question 6

```
function test(...args) {  
  console.log(args);  
}  
  
test();
```

Output কী?

53. Coding Challenge 1 — Sum

এই function তৈরি করো:

```
sum(10, 20, 30, 40);
```

Expected:

```
100
```

শুধু Rest Parameter ব্যবহার করে।

54. Coding Challenge 2 — Maximum

Function:

```
getMax(10, 50, 20, 80, 30);
```

Expected:

```
80
```

55. Coding Challenge 3 — Even Numbers

Function:

```
getEven(1, 2, 3, 4, 5, 6);
```

Expected:

```
[2, 4, 6]
```

56. Coding Challenge 4 — Average

Function:

```
average(10, 20, 30, 40);
```

Expected:

```
25
```

57. Coding Challenge 5 — First + Rest

Function:

```
process(10, 20, 30, 40);
```

এমনভাবে তৈরি করো যেন:

```
first = 10  
  
rest = [20, 30, 40]
```

58. Coding Challenge 6 — Shopping Cart

Function:

```
calculateTotal(500, 200, 300, 100);
```

Expected:

```
1100
```

Rest Parameter ব্যবহার করো।

59. Interview Questions

Q1. Rest parameter কী?

Function-এর remaining arguments-গুলোকে একটি array হিসেবে collect করার syntax।

```
function test(...args) {}
```

Q2. Rest parameter কি array?

হ্যাঁ।

```
function test(...args) {  
  console.log(Array.isArray(args));  
}
```

Result:

true

Q3. Rest parameter কোথায় থাকতে পারে?

Function parameter list-এর শেষে।

Q4. Rest vs Spread কী?

Rest:
multiple values → one array

Spread:
one iterable → multiple values

Q5. arguments কী?

Regular function-এর available একটি array-like object, যেখানে function-এ passed arguments access করা যায়।

Q6. arguments কি Array?

না।

এটি array-like object।

Q7. Arrow function-এর arguments আছে?

নিজস্ব arguments object নেই।

Q8. Modern JavaScript-এ arguments নাকি Rest Parameter prefer করা হয়?

নতুন code-এ সাধারণত **Rest Parameter** prefer করা হয়।

Q9. Rest Parameter কি empty হতে পারে?

হ্যাঁ।

```
function test(...args) {}  
  
test();
```

`args` হবে:

```
[]
```

Q10. একই function-এ Rest এবং Spread থাকতে পারে?

অবশ্যই।

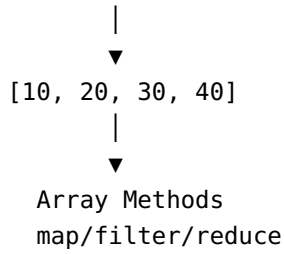
```
function getMax(...numbers) {  
  return Math.max(...numbers);  
}
```

প্রথমটি Rest, দ্বিতীয়টি Spread।

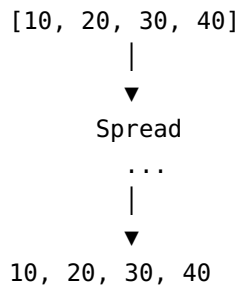
60. Mental Model

সবচেয়ে important diagram:

```
Function Call  
  
sum(10, 20, 30, 40)  
  |  
  ▼  
Rest Parameter  
  ...numbers
```



আর Spread:



61. Golden Rules ★

Rule 1

Rest parameter:

```
...args
```

arguments collect করে।

Rule 2

Rest parameter সবসময় last:

```
function test(a, b, ...rest) {}
```

Rule 3

Rest parameter একটি real Array।

```
Array.isArray(args); // true
```

Rule 4

`arguments` array-like object ।

```
Array.isArray(arguments); // false
```

Rule 5

Arrow function-এর নিজের `arguments` নেই ।

Rule 6

Modern code-এ variable number of arguments দরকার হলে Rest Parameter prefer করো ।

Rule 7

`...` দেখে শুধু syntax মুখস্থ করো না—**context** দেখো ।

```
function f(...args)
→ Rest

fn(...arr)
→ Spread

[...arr]
→ Spread

[a, ...rest]
→ Rest Pattern

{a, ...rest}
→ Rest Pattern
```

62. Final Mastery Checklist

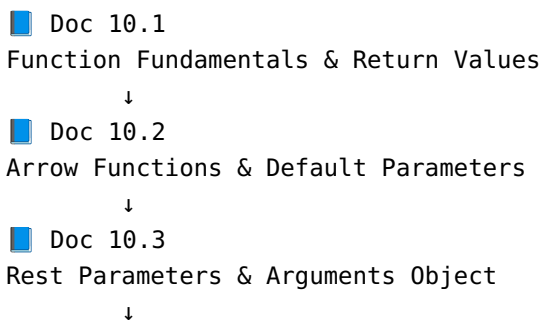
এই Doc শেষ করার পর তোমার পারা উচিত:

- ☐ Rest Parameter কী
- ☐ `...args`
- ☐ Variable number of arguments
- ☐ Rest parameter array হিসেবে কাজ করে
- ☐ Rest parameter empty হতে পারে
- ☐ Normal parameter + rest
- ☐ Rest parameter অবশ্যই last
- ☐ Rest + arrow function
- ☐ Rest + array methods
- ☐ `arguments` object
- ☐ `arguments.length`
- ☐ `arguments` array-like হওয়া
- ☐ `arguments` vs Array
- ☐ Arrow function-এর own `arguments` নেই
- ☐ `arguments` → Array conversion
- ☐ `Array.from(arguments)`
- ☐ `[...arguments]`
- ☐ Rest vs Spread
- ☐ Function-call spread
- ☐ Array spread
- ☐ Object rest-এর basic idea
- ☐ Rest/Spread context identify করা
- ☐ Modern JavaScript-এ Rest Parameter-এর advantage



Doc 10.3 Complete

এখন Functions section-এর flow:



■ Doc 10.4

IIFE & Callback Functions

Next এবং শেষ **Doc 10.4:**

IIFE + Callback Functions — এখানে callback কীভাবে function-এর argument হয়, synchronous callback, setTimeout callback, higher-order function-এর basic idea এবং IIFE scope/isolation—এসব একসাথে আসবে।